

Challenge Name: Volume Calculator

Description: Use function overloading to calculate the volume of different 3D shapes.

Problem Statement:

Volume is computed differently for cubes, spheres, and cylinders. Instead of creating multiple function names, function overloading allows a single volume() function with different parameter lists.

The program should support cube (side), sphere (radius), and cylinder (radius, height).

Input Format:

- Shape type and required parameters.

Constraints:

- Dimensions > 0

Output Format:

- Print volume.

Test Cases:

1. cube 3 $\rightarrow$ 27
2. sphere 2 $\rightarrow$ 33.51
3. cylinder 3 5 $\rightarrow$ 141.37

Code:

```
QA01.cpp U X
OOPS > WEEK 10 > QA01.cpp > main()
1 #include <iostream>
2 #include <cmath>
3 #include <omanip>
4 using namespace std;
5
6 double volume(double side) {
7     return pow(side, 3);
8 }
9
10 double volumeSphere(double radius) {
11     return (4.0 / 3.0) * M_PI * pow(radius, 3);
12 }
13
14 double volume(double radius, double height) {
15     return M_PI * pow(radius, 2) * height;
16 }
17
18 int main() {
19     string shape;
20     cin >> shape;
21
22     cout << fixed << setprecision(2);
23
24     if (shape == "cube") {
25         double side;
26         cin >> side;
27         cout << volume(side);
28     }
29     else if (shape == "sphere") {
30         double radius;
31         cin >> radius;
32         cout << volumeSphere(radius);
33     }
34     else if (shape == "cylinder") {
35         double radius, height;
36         cin >> radius >> height;
37         cout << volume(radius, height);
38     }
39     else {
40         cout << "Invalid shape type";
41     }
42
43     return 0;
44 }
45
```

Output:

```
PS K:\00PS\WEEK 10\QA01> .\QA01.exe
cube 3
27.00
PS K:\00PS\WEEK 10\QA01> .\QA01.exe
sphere 2
33.51
PS K:\00PS\WEEK 10\QA01> .\QA01.exe
cylinder 3 5
141.37
PS K:\00PS\WEEK 10\QA01> 
```

Challenge Name: Power Calculator

Description: Use function overloading to compute powers of integers and doubles.

Problem Statement:

Exponential calculations are frequent in mathematics. While integers and doubles may be handled separately, we can overload `power()` for both data types.

One function should take `(int, int)` while another takes `(double, int)`.

Input Format:

- Base and exponent.

Constraints:

- Exponent ≥ 0

Output Format:

- Print the result.

Test Cases:

1. `int 2 3` $\rightarrow$ 8
2. `double 2.5 2` $\rightarrow$ 6.25
3. `int 5 0` $\rightarrow$ 1

Code:

```
QA02.cpp U X
OPS > WEEK 10 > QA02 > QA02.cpp > main()
1  #include <iostream>
2  #include <cmath>
3  #include <iomanip>
4  using namespace std;
5
6  int power(int base, int exponent) {
7      return pow(base, exponent);
8  }
9
10 double power(double base, int exponent) {
11     return pow(base, exponent);
12 }
13
14 int main() {
15     string type;
16     cin >> type;
17
18     cout << fixed << setprecision(2);
19
20     if (type == "int") {
21         int base, exp;
22         cin >> base >> exp;
23         cout << power(base, exp);
24     }
25     else if (type == "double") {
26         double base;
27         int exp;
28         cin >> base >> exp;
29         cout << power(base, exp);
30     }
31     else {
32         cout << "Invalid type";
33     }
34
35     return 0;
36 }
```

Output:

```
PS K:\00PS\WEEK 10\QA02> .\QA02.exe
int 2 3
8
PS K:\00PS\WEEK 10\QA02> .\QA02.exe
double 2.5 2
6.25
PS K:\00PS\WEEK 10\QA02> .\QA02.exe
int 5 0
1
PS K:\00PS\WEEK 10\QA02> 
```

Challenge Name: Max Finder

Description: Overload functions to find maximum of two or three numbers.

Problem Statement:

Comparisons are common in applications like leaderboards or stock analysis. Overloading allows us to extend a single `maxValue()` function to handle two or three parameters.

You must implement `maxValue(int, int)` and `maxValue(int, int, int)`.

Input Format:

- Integers.

Constraints:

- Inputs within int range.

Output Format:

- Print maximum value.

Test Cases:

1. $2\ 5 \rightarrow 5$
2. $10\ 20\ 15 \rightarrow 20$
3. $7\ 7 \rightarrow 7$

Code:

```
OOPS > WEEK 10 > QA03 > QA03.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  int maxValue(int a, int b) {
5      return (a > b) ? a : b;
6  }
7
8  int maxValue(int a, int b, int c) {
9      return (a > b && a > c) ? a : (b > c ? b : c);
10 }
11
12 int main() {
13     int a, b, c;
14     string line;
15
16     getline(cin, line);
17     int count = 0;
18     for (char ch : line) {
19         if (ch == ' ') count++;
20     }
21
22     if (count == 1) {
23         sscanf(line.c_str(), "%d %d", &a, &b);
24         cout << maxValue(a, b);
25     }
26     else if (count == 2) {
27         sscanf(line.c_str(), "%d %d %d", &a, &b, &c);
28         cout << maxValue(a, b, c);
29     }
30     else {
31         cout << "Invalid input";
32     }
33
34     return 0;
35 }
36
```

Output:

```
PS K:\00PS\WEEK 10\QA03> .\QA03.exe
2 5
5
PS K:\00PS\WEEK 10\QA03> .\QA03.exe
10 20 15
20
PS K:\00PS\WEEK 10\QA03> .\QA03.exe
7 7
7
PS K:\00PS\WEEK 10\QA03> 
```

Challenge: Travel Package Planner

Problem Statement

Travel agencies often provide flexible packages for customers. Some customers may only book a destination, while others may book a destination along with the number of days, and a few may even include the type of accommodation they want. Constructor overloading helps manage these scenarios without writing separate classes for each case.

You are tasked with creating a `TravelPackage` class where:

- Constructor 1 accepts only the destination.
- Constructor 2 accepts destination and number of days.
- Constructor 3 accepts destination, days, and accommodation type.

This allows the system to create travel plans for customers with varying preferences. The class should include a method to display the complete package details.

Input Format

- Destination
- Number of days (or -1 if not provided)
- Accommodation type (or NA if not provided)

Constraints

- Destination: string (max length 50)
- Days: positive integer or -1
- Accommodation: string or NA

Output Format

- Display the travel package details.

Test Case 1

Paris

5

Hotel

Output:

Destination: Paris

Days: 5

Accommodation: Hotel

Test Case 2

Goa

-1

NA

Output:

Destination: Goa

Days: Not Provided

Accommodation: Not Provided

Test Case 3

Tokyo

7

NA

Output:

Destination: Tokyo

Days: 7

Accommodation: Not Provided

Code:

```
QA04.cpp U X
OOPS > WEEK 10 > QA04 > QA04.cpp > TravelPackage
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class TravelPackage {
6  private:
7      string destination;
8      int days;
9      string accommodation;
10
11 public:
12     TravelPackage(string dest) {
13         destination = dest;
14         days = -1;
15         accommodation = "NA";
16     }
17
18     TravelPackage(string dest, int d) {
19         destination = dest;
20         days = d;
21         accommodation = "NA";
22     }
23
24     TravelPackage(string dest, int d, string acc) {
25         destination = dest;
26         days = d;
27         accommodation = acc;
28     }
29
30     void display() {
31         cout << "Destination: " << destination << endl;
32         if (days == -1)
33             cout << "Days: Not Provided" << endl;
34         else
35             cout << "Days: " << days << endl;
36         if (accommodation == "NA")
37
38         class TravelPackage {
39             void display() {
40                 if (accommodation == "NA")
41                     cout << "Accommodation: Not Provided" << endl;
42                 else
43                     cout << "Accommodation: " << accommodation << endl;
44             }
45         };
46
47     int main() {
48         string destination;
49         int days;
50
51         getline(cin, destination);
52         cin >> days;
53         cin.ignore();
54         getline(cin, accommodation);
55
56         if (days == -1 && accommodation == "NA") {
57             TravelPackage tp(destination);
58             tp.display();
59         }
60         else if (accommodation == "NA") {
61             TravelPackage tp(destination, days);
62             tp.display();
63         }
64         else {
65             TravelPackage tp(destination, days, accommodation);
66             tp.display();
67         }
68         return 0;
69     }
```

Output:

```
PS K:\00PS\WEEK 10\QA04> .\QA04.exe
Paris
5
Hotel
Destination: Paris
Days: 5
Accommodation: Hotel
PS K:\00PS\WEEK 10\QA04> .\QA04.exe
Goa
-1
NA
Destination: Goa
Days: Not Provided
Accommodation: Not Provided
PS K:\00PS\WEEK 10\QA04> .\QA04.exe
Tokyo
7
NA
Destination: Tokyo
Days: 7
Accommodation: Not Provided
PS K:\00PS\WEEK 10\QA04> 
```

Gym Membership System

Problem Statement

Gyms often provide memberships with varying details. Some users register only with a name, others provide a name and duration in months, and some also include the type of plan (basic, premium). Instead of forcing customers to always provide full details, constructor overloading simplifies registration.

You need to implement a Membership class with three constructors:

- Name only
- Name + Duration
- Name + Duration + Plan

A method showMembership() should print the membership details.

Input Format

- Name
- Duration (or -1 if not provided)
- Plan (or NA if not provided)

Constraints

- Name: string (≤ 50 chars)
- Duration: integer ≥ 1 or -1
- Plan: string (≤ 20 chars) or NA

Output Format

- Display membership details.

Test Case 1

Karan

12

Premium

Output:

Name: Karan

Duration: 12 months

Plan: Premium

Test Case 2

Neha

-1

NA

Output:

Name: Neha

Duration: Not Provided

Plan: Not Provided

Test Case 3

Amit

6

NA

Output:

Name: Amit

Duration: 6 months

Plan: Not Provided

Code:

```
QA04.cpp U  QA05.cpp U  •
OOPS > WEEK 10 > QA05 > QA05.cpp > main()
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Membership {
6  private:
7      string name;
8      int duration;
9      string plan;
10
11 public:
12     // Constructor 1: Name only
13     Membership(string n) {
14         name = n;
15         duration = -1;
16         plan = "NA";
17     }
18
19     // Constructor 2: Name + Duration
20     Membership(string n, int d) {
21         name = n;
22         duration = d;
23         plan = "NA";
24     }
25
26     // Constructor 3: Name + Duration + Plan
27     Membership(string n, int d, string p) {
28         name = n;
29         duration = d;
30         plan = p;
31     }
32
33     void showMembership() {
34         cout << "Name: " << name << endl;
35
36         if (duration == -1)
37             cout << "Duration: Not Provided" << endl;
38
39         else
40             cout << "Duration: " << duration << " months" << endl;
41
42         if (plan == "NA")
43             cout << "Plan: Not Provided" << endl;
44         else
45             cout << "Plan: " << plan << endl;
46     }
47 };
48
49 int main() {
50     string name, plan;
51     int duration;
52     getline(cin, name);
53     cin >> duration;
54     cin.ignore();
55     getline(cin, plan);
56     if (duration == -1 && plan == "NA") {
57         Membership m(name);
58         m.showMembership();
59     }
60     else if (plan == "NA") {
61         Membership m(name, duration);
62         m.showMembership();
63     }
64     else {
65         Membership m(name, duration, plan);
66         m.showMembership();
67     }
68     return 0;
69 }
```

Output:

```
PS K:\00PS\WEEK 10\QA05> .\QA05.exe
Karan
12
Premium
Name: Karan
Duration: 12 months
Plan: Premium
PS K:\00PS\WEEK 10\QA05> .\QA05.exe
Meha
-1
NA
Name: Meh
Duration: Not Provided
Plan: Not Provided
PS K:\00PS\WEEK 10\QA05> .\QA05.exe
Amit
6
NA
Name: Amit
Duration: 6 months
Plan: Not Provided
PS K:\00PS\WEEK 10\QA05> 
```